claude-windows / dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3\subagents\workflows\wf_f1431155-4a0\agent-a485d30929a4e6914.jsonl`
- SHA-256: `11940f48b0cd66cfc578b82259b64f59f070083646b15bb7f87bd861636e532d`
- Source modified: `2026-06-21T07:27:10+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_f1431155-4a0`
- session_id: `dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3`

Transcript

1. user (2026-06-21T07:23:57.132Z)

```
You are reviewing milestone M1 of the COMPUTE_DECOUPLED_SERVING project in the repo at
C:/Users/avidu/Projects/badminton-highlight-indexer.
The spec is docs/COMPUTE_DECOUPLED_SERVING/README.md (read §2 D7-D15, §4.3 "Profile selection",
and §7 the tracker row M1). M1 must be: PURE, ADDITIVE, with deployment.profile='' (the default)
== today's behaviour, ZERO core change. A *deployment profile* is opt-in and applies a coherent
preset bundle of EXISTING knobs (indexing.detector_impl/skip_ai_handoff, storage.backend) plus
the
new deployment.compute_target. Precedence: built-in defaults < profile preset < config.json <
env.
D15: auto-detect SUGGESTS, never auto-applies (cloud spend never entered silently).

Files changed (read them all, plus the existing tests/test_config.py to understand the prior
contract):
backend/config/presets.py (NEW)
backend/config/models.py (added DeploymentConfig + AppConfig.deployment field)
backend/config/loader.py (added _deep_merge, _resolve_explicit_profile, preset application)
backend/config/__init__.py (re-exports)
tests/test_config_deployment.py (NEW)

Ground EVERY claim in the actual code with a file:line citation. Do NOT speculate. If you assert
a
bug, give the exact input that triggers it and the wrong-vs-right output. Rate confidence
honestly;
prefer FEWER, HIGH-CONFIDENCE findings. A clean bill ([] findings) is a valid and good result.

LENS: precedence & deep-merge correctness. Trace load_config in backend/config/loader.py.
Verify defaults < preset < config.json < env(DEPLOYMENT_PROFILE) holds for EVERY combination.
Probe:
(a) does a config.json sub-key override a preset sub-key while a SIBLING preset sub-key in the
same
section survives? (b) the env-wins-over-file profile re-stamp ? is the recorded profile honest
and is
compute_target consistent? (c) the compute_target mismatch warning path; (d) _deep_merge
recursing into
free-form dict fields (storage.local/s3/gcs/gdrive, fusion.cue_flags) ? could merging-not-
replacing a
populated dict ever surprise a profile user? (e) is there ANY case where the no-profile path is
NOT the
exact original {**defaults, **file_data}?
```

2. assistant (2026-06-21T07:23:59.508Z)

I'll start by reading the spec and then the changed files to understand the contract.

3. user (2026-06-21T07:24:03.004Z)

```
1       """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3       A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4       config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5       knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7       Precedence (lowest ? highest), realised by the loader::
8
9           built-in model defaults  <  profile preset  <  config.json  <  env  <  worker --set
10
11      This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12      module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13      (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14      profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15      today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16      spend is never entered silently).
17      """
18
19      from __future__ import annotations
20
21      import copy
22      import os
23      from typing import Any, Mapping, Optional
24
25      # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26      # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27      PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28      COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30      # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31      # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32      COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33          "truly-local": "local_gpu",
34          "cloud-serving": "cloud_run",
35          "cpu-mock": "cpu_mock",
36      }
37
38      # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39      # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40      # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41      # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42      # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43      PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44          "truly-local": {
45              "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46              # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47              # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48              "indexing": {"skip_ai_handoff": True},
49              "storage": {"backend": "local"},
50          },
51          "cloud-serving": {
52              "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53              # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
```

```
(D11).
54              "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55              "storage": {"backend": "gcs"},
56          },
57          "cpu-mock": {
58              "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59              # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60              "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
61              "storage": {"backend": "local"},
62          },
63      }
64
65
66      def is_known_profile(profile: str) -> bool:
67          """True for a non-empty, recognised profile name."""
68          return bool(profile) and profile in PROFILE_PRESETS
69
70
71      def preset_for(profile: str) -> dict[str, Any]:
72          """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
73          module-level table), or ``{}`` for ``""`` / an unknown profile."""
74          return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
75
76
77      def _truthy(val: Optional[str]) -> bool:
78          """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
79          if val is None:
80              return False
81          return val.strip().lower() in {"1", "true", "yes", "on"}
82
83
84      def _cuda_available() -> bool:
85          """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
86          ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
87          try:
88              # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
89              # and optional). Only reached when a caller opts into probe_cuda.
90              import torch
91
92              return bool(torch.cuda.is_available())
93          except Exception:
94              return False
95
96
97      def suggest_profile(
98          env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
99      ) -> str:
100         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
101
102         Signals, in priority order:
103
104           - ``K_SERVICE`` set (Cloud Run)        ? ``"cloud-serving"``
105           - ``CI`` truthy                        ? ``"cpu-mock"``
106           - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
107           - otherwise                            ? ``""`` (no strong suggestion)
108
109         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
110         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
```

```
111         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
112         """
113         e = os.environ if env is None else env
114         if e.get("K_SERVICE"):
115             return "cloud-serving"
116         if _truthy(e.get("CI")):
117             return "cpu-mock"
118         if probe_cuda and _cuda_available():
119             return "truly-local"
120         return ""
121
```

4. user (2026-06-21T07:24:03.058Z)

```
1       """Configuration loader and saver.
2
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8
9       from __future__ import annotations
10
11      import json
12      import logging
13      import os
14      from pathlib import Path
15      from typing import Any
16
17      from pydantic import BaseModel
18
19      from .models import AppConfig
20      from .presets import (
21          COMPUTE_TARGET_FOR_PROFILE,
22          PROFILE_PRESETS,
23          is_known_profile,
24          preset_for,
25          suggest_profile,
26      )
27
28      logger = logging.getLogger(__name__)
29
30      DEFAULT_CONFIG_PATH = Path("config.json")
31
32
33      def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
34          """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
scalars
35          replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
36          overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
profile
37          path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
38          out = dict(base)
39          for key, val in override.items():
40              if isinstance(val, dict) and isinstance(out.get(key), dict):
41                  out[key] = _deep_merge(out[key], val)
42              else:
43                  out[key] = val
44          return out
45
46
47      def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
48          """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
```

```
none
49          is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
a bare
50          environment never silently selects a profile (and never silently spends on
cloud)."""
51          env_profile = os.environ.get("DEPLOYMENT_PROFILE")
52          if env_profile and env_profile.strip():
53              return env_profile.strip()
54          dep = file_data.get("deployment")
55          if isinstance(dep, dict):
56              prof = dep.get("profile")
57              if isinstance(prof, str) and prof.strip():
58                  return prof.strip()
59          return ""
60
61
62      def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"") -> list[str]:
63          """Dotted paths in `data` that the typed config will not honor.
64
65          Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
66          kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
67          typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
68          user's intent is lost ? collect every such key so load_config can warn.
69          """
70          unknown: list[str] = []
71          fields = model_cls.model_fields
72          for key, val in data.items():
73              if key not in fields:
74                  unknown.append(prefix + key)
75                  continue
76              ann = fields[key].annotation
77              if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
78                  unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
79          return unknown
80
81
82      def _get_builtin_defaults() -> dict[str, Any]:
83          """Return a plain dict of the current built-in defaults.
84
85          The AppConfig model is the single source of truth for defaults.
86          """
87          return AppConfig().model_dump(mode="json")
88
89
90      def load_config(path: str | Path | None = None) -> AppConfig:
91          """Load configuration with the following precedence:
92
93          1. Built-in defaults defined in the Pydantic models.
94          2. Values from the JSON file (if present and readable).
95          3. (Future) environment / CLI overrides.
96
97          Missing file or unreadable file ? returns a fully defaulted AppConfig.
98          Unknown keys in the file are tolerated (extra="allow" on the model).
99          """
100         cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
101
102         file_data: dict[str, Any] = {}
103         if cfg_path.exists():
104             try:
105                 with open(cfg_path, "r", encoding="utf-8") as f:
106                     file_data = json.load(f)
107             except Exception as exc:
108                 # Non-fatal: continue with defaults + whatever we could parse.
```

```
109                 # In production we might log here.
110                 logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
111
112         # A config.json that is valid JSON but not an OBJECT (e.g. `[]`, `42`, `"x"`,
`null`) would
113         # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
114         # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
warn, per the
115         # loader's "bad input is non-fatal" contract.
116         if not isinstance(file_data, dict):
117             logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
118                            cfg_path, type(file_data).__name__)
119             file_data = {}
120
121         if file_data:
122             unknown = _unknown_key_paths(file_data, AppConfig)
123             if unknown:
124                 logger.warning(
125                     "[CONFIG WARNING] %s contains keys the typed config does not "
126                     "recognize (typo'd or removed knobs ? top-level extras are kept "
127                     "but unread; unknown section keys are silently dropped): %s",
128                     cfg_path, ", ".join(sorted(unknown)),
129                 )
130
131         # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
at
132         # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
133         # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
134         # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
135         defaults = _get_builtin_defaults()
136         profile = _resolve_explicit_profile(file_data)
137
138         if profile and not is_known_profile(profile):
139             # An unrecognised EXPLICIT profile is loud, not silent (the loader's "dropped
config"
140             # ethos): warn and apply no preset. A bad value coming from config.json
additionally
141             # hits the typed Literal at model_validate below (a hard, clear error); a bad
value
142             # coming only from the env override is ignored here with this warning.
143             logger.warning(
144                 "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
145                 profile, ", ".join(PROFILE_PRESETS),
146             )
147             profile = ""
148
149         if profile:
150             merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
151             # Record the profile actually applied ? the env may have chosen it over
config.json,
152             # so re-stamp it onto the merged deployment section to keep the record honest.
153             merged.setdefault("deployment", {})
154             if isinstance(merged["deployment"], dict):
155                 merged["deployment"]["profile"] = profile
156                 ct = merged["deployment"].get("compute_target")
157                 canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
158                 if ct and canonical and ct != canonical:
159                     logger.warning(
```

```
160                         "[CONFIG] deployment profile %r normally uses compute_target=%r but
"
161                         "config.json overrides it to %r ? honoring the explicit override.",
162                         profile, canonical, ct,
163                     )
164                 logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
165                             profile, merged["deployment"].get("compute_target"))
166         else:
167             # No profile ? the original shallow merge (Pydantic re-defaults nested
sections).
168             merged = {**defaults, **file_data}
169             suggestion = suggest_profile()  # cheap, env-only (no torch); SUGGESTS only
(D15)
170             if suggestion:
171                 logger.debug(
172                     "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
173                     "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
174                     "auto-applies ? D15).", suggestion,
175                 )
176
177         return AppConfig.model_validate(merged)
178
179
180     def save_default_config(path: str | Path | None = None) -> Path:
181         """Write a fresh default config.json.
182
183         Used by scripts/doctor.py diagnostics and the --setup CLI path.
184         Overwrites if the file exists.
185         """
186         cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
187         cfg = AppConfig()
188         cfg_path.parent.mkdir(parents=True, exist_ok=True)
189         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
190         return cfg_path
191
192
193     # Backwards-compatible alias used by some older call sites during transition.
194     def get_default_config() -> dict[str, Any]:
195         """Return defaults as a plain dict (for transitional use only)."""
196         return _get_builtin_defaults()
197
```

5. user (2026-06-21T07:24:03.115Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
```

```
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43          velocity_threshold: float = 0.02
44          # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45          # decoded frame cache) are deleted automatically ? they are pure intermediates,
46          # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47          # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48          keep_frames: bool = False
49          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50          # 0 = disabled.
51          min_free_gb: float = 0.0
52
53
54      class WasbIndexConfig(BaseModel):
55          """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57          Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58          these knobs actually take effect ? previously this whole block was silently dropped
59          because nested config sections defaulted to extra='ignore'.
60          """
61          wsl_distro: str = "Ubuntu"
62          repo_dir: str = "~/models/WASB-SBDT"
63          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64          conda_env: str = "wasb"
65          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66          sport: str = "badminton"
67          wsl_stage_dir: str = "~/clips"
68          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69          velocity_threshold: float = 0.02
70          # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
71          # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
72          device: str = "cuda"        # torch device for the native runner: cuda | mps | cpu
73          python_bin: str = "python"  # the `wasb` conda env python (invoked by path, no
`conda activate`)
74          # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
75          # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
76          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
77          keep_frames: bool = False
78          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
```

```
disabled.
79          min_free_gb: float = 0.0
80          # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
81          # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
82          # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
83          # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
84          # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
85          # Tri-state:
86          #   None       = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
87          #                behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
88          #   True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
89          stream_video: Optional[bool] = None
90
91
92      class FusionConfig(BaseModel):
93          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
94
95          Every default reproduces control behaviour: the fusion segmenter persists the
96          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
97          true ? the person-cue features, but it does NOT gate the served handoff unless a
98          threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
99          is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
100         supplied ? off-court persons (spectators / adjacent courts) are excluded.
101         """
102         # Which trajectory substrate seeds the windows (shuttle stays primary).
103         substrate: Literal["wasb", "tracknet"] = "wasb"
104         # Windowing velocity threshold for the fusion family (the engine reads
105         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
106         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
107         velocity_threshold: float = 0.02
108         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
109         # enabled ? so the default path never imports torch / touches the GPU.
110         cue_flags: dict[str, bool] = Field(default_factory=dict)
111         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
112         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
113         min_crossings: int = Field(default=0, ge=0)
114         # Served Gate B: when the person cue is on, drop windows with median in-court
players
115         # < this. 0 = OFF.
116         min_players: int = Field(default=0, ge=0)
117         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
118         court_polygon: Optional[list[list[float]]] = None
119         # Net line for the person two-sided-motion split (person features only ? the shuttle
120         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
121         net_axis: Literal["x", "y"] = "y"
122         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
123
124
125     class IndexingConfig(BaseModel):
126         default_segmenter: str = "yolo_hybrid"
127         use_gpu: bool = True
128         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
```

```
129         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
130         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
131         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
132         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
133         detector_impl: str = "auto"
134         motion_threshold: float = 0.08
135         min_segment_duration: float = 3.0
136         dead_time_merge_gap: float = 2.0
137         chunk_duration: float = 90.0
138         chunk_overlap: float = 10.0
139         detector_model: str = "fasterrcnn_resnet50_fpn"
140         detector_score_threshold: float = 0.5
141         yolo_velocity_threshold: float = 0.15
142         yolo_frame_skip: int = 3
143         yolo_tracker: str = "bytetrack.yaml"
144         yolo_prefetch_workers: int = 2
145         min_action_window_duration: float = 2.0
146         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
147         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
148         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
149         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
150         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
151         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
152         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
153         max_window_duration: float = 6.0
154         window_min_split_gap: float = 0.5  # min internal gap (s) that qualifies as a split
point
155         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
156         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
157         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
158         # Only cuts (recall can't drop). OFF by default until measured/promoted.
159         window_hard_cap: bool = False
160         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
161         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
162         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
163         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
164         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
165         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
166         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
167         window_inactivity_end: float = 1.5
168         inactivity_rally_thresh: float = 0.20
169         inactivity_min_density: float = 0.30
170         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
171         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
172         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
```

```
signal ? e.g.
173         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
174         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
175         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
176         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
177         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
178         window_blob_fallback: bool = False
179         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
180         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
181         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
182         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
183         window_blob_factor: float = 4.0
184         log_yolo_candidates: bool = True
185         store_yolo_candidates: bool = True
186         ai_handoff_padding: float = 2.0
187         ai_max_workers: int = 5
188         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
189         # chunks of a high-bitrate (4K) source blow past the provider upload cap
190         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
191         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
192         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
193         # Matches gemini_refine's --clip-scale-width default.
194         ai_chunk_scale_width: int = 1280
195         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
196         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
197         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
198         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
199         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
200         skip_ai_handoff: bool = False
201         # Optional debug knob: trim every video to the first N seconds before processing.
202         debug_duration: Optional[float] = None
203
204         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
205         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
206         fusion: FusionConfig = Field(default_factory=FusionConfig)
207
208         @field_validator("default_segmenter")
209         @classmethod
210         def segmenter_must_be_registered(cls, v: str) -> str:
211             # Registry check happens at runtime in main/segmenter selection.
212             # Here we just allow any string for forward compatibility.
213             return v
214
215         @model_validator(mode="after")
216         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
217             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
218             # step makes `while t < duration: t += step` loop forever, growing memory before
any
219             # GPU work. Reject the bad config at load time instead.
220             if self.chunk_overlap >= self.chunk_duration:
221                 raise ValueError(
222                     f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
```

```
"
                    f"({self.chunk_duration}); otherwise chunking never advances."
                )
            return self


    class OutputConfig(BaseModel):
        default_highlights_name: str = "highlights.mp4"
        db_name: str = "sports_indexer.db"
        # Directory where the DB, highlight reels, and processed clips are written.
        # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
        output_dir: str = "output"


    class WatermarkConfig(BaseModel):
        """Branding overlay burned into each highlight clip during the per-clip re-encode
        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
        (under `stitching.watermark` in config.json) to turn it off. The text is fully
        configurable. The concat stage stays stream-copy (-c copy)."""
        enabled: bool = True
        text: str = "Generated by Khelsutra.guru"
        logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
        font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
        font: str = "Sans"         # fontconfig family used when font_path is empty
        font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
        opacity: float = Field(default=0.85, ge=0.0, le=1.0)
        box: bool = True           # faint backing box behind the text for legibility
        margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
        position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"


    class StitchingConfig(BaseModel):
        begin_padding: float = 0.5
        end_padding: float = 0.5
        use_cache: bool = False
        min_shot_count: int = 1
        # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
        # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
        # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
        # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
        # local detector over-fires and its false positives are mostly SHORT (motion blips,
        # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
        # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
        min_rally_duration: float = Field(default=0.0, ge=0.0)
        watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


    class LoggingConfig(BaseModel):
        """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
        # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
        # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
        # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
        # them to WARNING; set true to restore full chatter for request-flow debugging.
        verbose_http: bool = False


    class SecurityConfig(BaseModel):
```

```
279         # When set, /api/process video_path must resolve under this directory (hosted/tenant
280         # mode). Default None preserves the single-user local 'any local file' workflow.
281         allowed_video_root: Optional[str] = None
282
283         # --- Chunked upload (B2, PR-2) ---
284         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
285         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
286         # contain upload_dir or /api/process will reject the uploaded paths.
287         upload_dir: str = "output/uploads"
288         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
289         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
290         max_upload_mb: int = 4096
291         max_part_mb: int = 95
292         allowed_upload_extensions: List[str] = ["mp4", "mov"]
293
294
295     class StorageConfig(BaseModel):
296         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
297         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
298         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
299         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
300         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
301         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
302         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
303         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
304         output_root: str = ""
305         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
306         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
307         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
308         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
309         # the legacy flat layout is used unchanged.
310         single_folder_per_video: bool = False
311         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
312         workspace_root: str = ""
313         # Cloud namespace prefix under the root so per-video folders sit tidily beside
314         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
315         workspace_prefix: str = "workspaces"
316         local: dict[str, Any] = Field(default_factory=dict)
317         s3: dict[str, Any] = Field(default_factory=dict)
318         gcs: dict[str, Any] = Field(default_factory=dict)
319         gdrive: dict[str, Any] = Field(default_factory=dict)
320
321
322     class DeploymentConfig(BaseModel):
323         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
324         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
325
326         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
327         behaves exactly as before this section existed. A profile is opt-in (``config.json``
328         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
329         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
330         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
```

```
knobs
331         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
332
333         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
334
335         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
336         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
337         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
338         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
339         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
340
341
342     class AppConfig(BaseModel):
343         """Root configuration object.
344
345         All runtime configuration should be accessed via an instance of this class
346         (or the loader) rather than raw dict lookups.
347         """
348
349         sport: Sport = "badminton"
350         ai_provider: str = "gemini"
351         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
352         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
353         ai_model: str = "gemini-flash-latest"
354
355         validation: ValidationConfig = Field(default_factory=ValidationConfig)
356         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
357         output: OutputConfig = Field(default_factory=OutputConfig)
358         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
359         security: SecurityConfig = Field(default_factory=SecurityConfig)
360         logging: LoggingConfig = Field(default_factory=LoggingConfig)
361         storage: StorageConfig = Field(default_factory=StorageConfig)
362         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
363
364         # Allow extra keys during transition so existing config.json files do not break.
365         model_config = {
366             "extra": "allow",
367             "validate_assignment": True,
368         }
369
370         # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
371         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
372         # attribute access / the typed config. A nested *section* is returned as a plain
dict
373         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
374         # section model instead breaks it (a model has no .get), which bit every validator
on
375         # real runs. Scalar leaves are returned as-is.
376         def get(self, key: str, default: Any = None) -> Any:
377             _MISSING = object()
378             val: Any = _MISSING
379             if key in type(self).model_fields or hasattr(self, key):
380                 val = getattr(self, key, _MISSING)
381             if val is _MISSING:
382                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
383                 # discovered dynamically from the model fields, so a future top-level
section
384                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
```

```
                for fname in type(self).model_fields:
                    section = getattr(self, fname, None)
                    if isinstance(section, BaseModel) and hasattr(section, key):
                        val = getattr(section, key)
                        break
            if val is _MISSING:
                return default
            if isinstance(val, BaseModel):
                return val.model_dump(mode="json")
            return val

        def __getitem__(self, key: str) -> Any:
            val = self.get(key)
            if val is None and key not in type(self).model_fields:
                # Re-raise like a real dict for unknown top-level
                raise KeyError(key)
            return val


    # Convenience type alias for callers that want to annotate
    Config = AppConfig

```

6. user (2026-06-21T07:24:03.193Z)

```
"""Configuration package.

Public API:
    from backend.config import load_config, AppConfig, Config

This package is the canonical place for all configuration access.
"""

from .loader import load_config, save_default_config
from .models import (  # re-export common ones
    AppConfig,
    Config,
    DeploymentConfig,
    IndexingConfig,
    StitchingConfig,
    StorageConfig,
    ValidationConfig,
)
from .presets import PROFILE_PRESETS, suggest_profile

__all__ = [
    "load_config",
    "save_default_config",
    "AppConfig",
    "Config",
    "DeploymentConfig",
    "IndexingConfig",
    "StitchingConfig",
    "StorageConfig",
    "ValidationConfig",
    "PROFILE_PRESETS",
    "suggest_profile",
]

```

7. user (2026-06-21T07:24:03.267Z)

```
"""Tests for the deployment-profile config (COMPUTE_DECOUPLED_SERVING M1).

What this pins:
```

```
4       - The default profile ("") applies NO preset ? the loaded config is byte-identical to
the
5         pre-M1 default, so the core is unaffected (the project's "default-OFF, zero core
change").
6       - Each profile applies its coherent preset bundle (compute_target + the existing
7         detector_impl / skip_ai_handoff / storage.backend knobs).
8       - Precedence: built-in defaults < profile preset < config.json (a file sub-key overrides
a
9         preset sub-key while sibling preset/default sub-keys survive ? i.e. real deep-merge
layering).
10      - D15: auto-detect SUGGESTS but never auto-applies (a bare cloud-like env does NOT
silently
11        select a profile / spend on cloud).
12      - The model's Literal enum stays in parity with backend/config/presets.PROFILE_PRESETS.
13      """
14
15      import json
16      import logging
17      import os
18
19      import pytest
20      from pydantic import ValidationError
21
22      from backend.config import AppConfig, DeploymentConfig, load_config, save_default_config
23      from backend.config.presets import (
24          COMPUTE_TARGET_FOR_PROFILE,
25          COMPUTE_TARGETS,
26          PROFILE_PRESETS,
27          PROFILES,
28          is_known_profile,
29          preset_for,
30          suggest_profile,
31      )
32
33      NONEXISTENT = "/nonexistent/path/12345/deployment_config.json"
34
35
36      def _write(tmp_path, data) -> str:
37          p = tmp_path / "cfg.json"
38          p.write_text(json.dumps(data))
39          return str(p)
40
41
42      # ---------------------------------------------------------------------------------
43      # Defaults / additivity ? the default path must be unchanged.
44      # ---------------------------------------------------------------------------------
45      def test_deployment_defaults_are_empty_and_inert():
46          cfg = load_config(NONEXISTENT)
47          assert isinstance(cfg.deployment, DeploymentConfig)
48          assert cfg.deployment.profile == ""
49          assert cfg.deployment.compute_target == ""
50
51
52      def test_default_path_applies_no_preset(monkeypatch):
53          """With no profile selected, the existing knobs keep their built-in defaults (no
preset
54          mutates detector_impl / skip_ai_handoff / storage.backend)."""
55          monkeypatch.delenv("DEPLOYMENT_PROFILE", raising=False)
56          cfg = load_config(NONEXISTENT)
57          assert cfg.indexing.detector_impl == "auto"
58          assert cfg.indexing.skip_ai_handoff is False
59          assert cfg.storage.backend == "local"
60
61
62      def test_default_config_dump_only_adds_deployment_section():
```

```
        """Adding the section must not perturb any other default ? the dump equals a fresh
        AppConfig()'s dump, and the new section is present with empty values."""
        dumped = load_config(NONEXISTENT).model_dump(mode="json")
        assert dumped == AppConfig().model_dump(mode="json")
        assert dumped["deployment"] == {"profile": "", "compute_target": ""}


    # ---------------------------------------------------------------------------------------
    # Parity: the model Literal must match the presets table (no drift).
    # ---------------------------------------------------------------------------------------
    def test_profile_literal_matches_presets():
        import typing

        literal_profiles =
set(typing.get_args(DeploymentConfig.model_fields["profile"].annotation))
        assert literal_profiles == {"", *PROFILES}
        assert set(PROFILE_PRESETS) == set(PROFILES)
        assert set(COMPUTE_TARGET_FOR_PROFILE) == set(PROFILES)


    def test_compute_target_literal_matches_presets():
        import typing

        literal_targets =
set(typing.get_args(DeploymentConfig.model_fields["compute_target"].annotation))
        assert literal_targets == {"", *COMPUTE_TARGETS}
        assert set(COMPUTE_TARGET_FOR_PROFILE.values()) == set(COMPUTE_TARGETS)


    def test_each_preset_only_touches_existing_knobs():
        """A preset must only set knobs that exist on the typed models ? otherwise it would
be a
        silent no-op. Validates the merged result against AppConfig for every profile."""
        for profile in PROFILES:
            # A preset's value set must validate cleanly when merged onto defaults.
            merged = {**AppConfig().model_dump(mode="json")}
            for section, kv in preset_for(profile).items():
                merged.setdefault(section, {})
                if isinstance(merged[section], dict):
                    merged[section].update(kv)
            AppConfig.model_validate(merged)  # raises if a preset key/value is invalid


    # ---------------------------------------------------------------------------------------
    # Per-profile preset application.
    # ---------------------------------------------------------------------------------------
    def test_cpu_mock_profile(tmp_path):
        cfg = load_config(_write(tmp_path, {"deployment": {"profile": "cpu-mock"}}))
        assert cfg.deployment.profile == "cpu-mock"
        assert cfg.deployment.compute_target == "cpu_mock"
        assert cfg.indexing.detector_impl == "stub"
        assert cfg.indexing.skip_ai_handoff is True
        assert cfg.storage.backend == "local"


    def test_cloud_serving_profile(tmp_path):
        cfg = load_config(_write(tmp_path, {"deployment": {"profile": "cloud-serving"}}))
        assert cfg.deployment.profile == "cloud-serving"
        assert cfg.deployment.compute_target == "cloud_run"
        assert cfg.indexing.detector_impl == "native"
        assert cfg.indexing.skip_ai_handoff is False  # Gemini-ON until the owned model
clears the floor
        assert cfg.storage.backend == "gcs"


```

```
    def test_truly_local_profile(tmp_path):
        cfg = load_config(_write(tmp_path, {"deployment": {"profile": "truly-local"}}))
        assert cfg.deployment.profile == "truly-local"
        assert cfg.deployment.compute_target == "local_gpu"
        # detector_impl is intentionally left at the production default (env picks WSL vs
native).
        assert cfg.indexing.detector_impl == "auto"
        assert cfg.indexing.skip_ai_handoff is True  # zero-cloud
        assert cfg.storage.backend == "local"


    # ---------------------------------------------------------------------------------------
    # Precedence: defaults < preset < config.json (deep-merge layering).
    # ---------------------------------------------------------------------------------------
    def test_config_json_overrides_preset_subkey(tmp_path):
        """A file sub-key beats the preset's sub-key, but a sibling preset sub-key in the
SAME
        section survives (this is the deep-merge layering ? a shallow merge would wipe
it)."""
        cfg = load_config(_write(tmp_path, {
            "deployment": {"profile": "cpu-mock"},
            "indexing": {"detector_impl": "native"},  # override one preset knob...
        }))
        assert cfg.indexing.detector_impl == "native"     # file wins
        assert cfg.indexing.skip_ai_handoff is True       # ...preset sibling SURVIVES (deep
merge)


    def test_preset_does_not_wipe_unrelated_defaults(tmp_path):
        """Selecting a profile + setting an unrelated knob keeps the rest of the section at
its
        built-in default (the preset and the file both layer onto the full defaults)."""
        cfg = load_config(_write(tmp_path, {
            "deployment": {"profile": "cpu-mock"},
            "indexing": {"ai_max_workers": 2},
        }))
        assert cfg.indexing.detector_impl == "stub"   # preset
        assert cfg.indexing.skip_ai_handoff is True   # preset
        assert cfg.indexing.ai_max_workers == 2       # file
        assert cfg.indexing.default_segmenter == "yolo_hybrid"  # untouched default
        assert cfg.indexing.max_window_duration == 6.0          # untouched default


    def test_config_json_can_override_compute_target_with_warning(tmp_path, caplog):
        cfg_path = _write(tmp_path, {
            "deployment": {"profile": "cpu-mock", "compute_target": "cloud_run"},
        })
        with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
            cfg = load_config(cfg_path)
        assert cfg.deployment.profile == "cpu-mock"
        assert cfg.deployment.compute_target == "cloud_run"  # explicit override honored
(D15)
        assert any("overrides it to 'cloud_run'" in r.message for r in caplog.records)


    # ---------------------------------------------------------------------------------------
    # Env selection + precedence over config.json.
    # ---------------------------------------------------------------------------------------
    def test_env_selects_profile(tmp_path, monkeypatch):
        monkeypatch.setenv("DEPLOYMENT_PROFILE", "cpu-mock")
        cfg = load_config(_write(tmp_path, {}))
        assert cfg.deployment.profile == "cpu-mock"
        assert cfg.indexing.detector_impl == "stub"


```

```
    def test_env_profile_overrides_config_json_profile(tmp_path, monkeypatch):
        """env wins over config.json for the profile CHOICE, and the recorded profile
matches the
        preset actually applied (not the file's value)."""
        monkeypatch.setenv("DEPLOYMENT_PROFILE", "cpu-mock")
        cfg = load_config(_write(tmp_path, {"deployment": {"profile": "cloud-serving"}}))
        assert cfg.deployment.profile == "cpu-mock"        # env value recorded
        assert cfg.deployment.compute_target == "cpu_mock"  # cpu-mock preset applied
        assert cfg.indexing.detector_impl == "stub"


    def test_blank_env_profile_is_ignored(tmp_path, monkeypatch):
        monkeypatch.setenv("DEPLOYMENT_PROFILE", "   ")
        cfg = load_config(_write(tmp_path, {}))
        assert cfg.deployment.profile == ""
        assert cfg.indexing.detector_impl == "auto"


    def test_unknown_env_profile_warns_and_applies_no_preset(tmp_path, monkeypatch, caplog):
        monkeypatch.setenv("DEPLOYMENT_PROFILE", "banana")
        with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
            cfg = load_config(_write(tmp_path, {}))
        assert cfg.deployment.profile == ""          # not applied
        assert cfg.indexing.detector_impl == "auto"  # no preset
        assert any("unrecognized deployment profile 'banana'" in r.message for r in
caplog.records)


    def test_unknown_profile_in_config_json_raises(tmp_path, monkeypatch):
        """A bad VALUE in config.json hits the typed Literal ? a loud, hard error (same
class as a
        bad `sport`), not a silent drop."""
        monkeypatch.delenv("DEPLOYMENT_PROFILE", raising=False)
        with pytest.raises(ValidationError):
            load_config(_write(tmp_path, {"deployment": {"profile": "banana"}}))


    # ------------------------------------------------------------------------------------
    # D15 ? auto-detect SUGGESTS but never AUTO-APPLIES.
    # ------------------------------------------------------------------------------------
    def test_cloud_env_does_not_auto_apply_profile(tmp_path, monkeypatch):
        """K_SERVICE (Cloud Run) makes suggest_profile() recommend cloud-serving, but with
no
        explicit profile the loaded config stays fully default ? NO silent cloud
selection."""
        monkeypatch.delenv("DEPLOYMENT_PROFILE", raising=False)
        monkeypatch.setenv("K_SERVICE", "khelsutra-serving")
        cfg = load_config(_write(tmp_path, {}))
        assert cfg.deployment.profile == ""           # NOT auto-applied
        assert cfg.deployment.compute_target == ""
        assert cfg.indexing.detector_impl == "auto"   # default, not "native"
        assert cfg.storage.backend == "local"         # default, not "gcs"


    def test_active_profile_is_logged(tmp_path, caplog):
        with caplog.at_level(logging.INFO, logger="backend.config.loader"):
            load_config(_write(tmp_path, {"deployment": {"profile": "truly-local"}}))
        assert any("deployment profile ACTIVE: truly-local" in r.message for r in
caplog.records)


    def test_no_active_profile_info_when_unset(tmp_path, caplog, monkeypatch):
        monkeypatch.delenv("DEPLOYMENT_PROFILE", raising=False)
        with caplog.at_level(logging.INFO, logger="backend.config.loader"):
            load_config(_write(tmp_path, {}))
```

```
        assert not any("profile ACTIVE" in r.message for r in caplog.records)


    # ------------------------------------------------------------------------------
    # suggest_profile() ? pure env-only logic + the opt-in cuda probe.
    # ------------------------------------------------------------------------------
    def test_suggest_profile_cloud_run():
        assert suggest_profile({"K_SERVICE": "svc"}) == "cloud-serving"


    def test_suggest_profile_ci():
        assert suggest_profile({"CI": "true"}) == "cpu-mock"
        assert suggest_profile({"CI": "1"}) == "cpu-mock"
        assert suggest_profile({"CI": "yes"}) == "cpu-mock"


    def test_suggest_profile_ci_falsy_or_absent():
        assert suggest_profile({}) == ""
        assert suggest_profile({"CI": "false"}) == ""
        assert suggest_profile({"CI": "0"}) == ""
        assert suggest_profile({"CI": ""}) == ""


    def test_suggest_profile_k_service_beats_ci():
        assert suggest_profile({"K_SERVICE": "svc", "CI": "true"}) == "cloud-serving"


    def test_suggest_profile_no_cuda_probe_by_default(monkeypatch):
        """probe_cuda defaults False ? no torch import, no truly-local suggestion even with
a GPU."""
        import backend.config.presets as presets

        monkeypatch.setattr(presets, "_cuda_available", lambda: True)
        assert suggest_profile({}) == ""                    # probe off ? no suggestion
        assert suggest_profile({}, probe_cuda=True) == "truly-local"


    def test_suggest_profile_cuda_probe_false(monkeypatch):
        import backend.config.presets as presets

        monkeypatch.setattr(presets, "_cuda_available", lambda: False)
        assert suggest_profile({}, probe_cuda=True) == ""


    def test_cuda_available_never_raises(monkeypatch):
        """Even a broken/missing torch must degrade to False, never propagate."""
        import builtins

        import backend.config.presets as presets

        real_import = builtins.__import__

        def boom(name, *a, **k):
            if name == "torch":
                raise RuntimeError("no torch")
            return real_import(name, *a, **k)

        monkeypatch.setattr(builtins, "__import__", boom)
        assert presets._cuda_available() is False


    # ------------------------------------------------------------------------------
    # Helpers + plumbing.
    # ------------------------------------------------------------------------------
    def test_is_known_profile():
```

```
306         assert is_known_profile("cpu-mock") is True
307         assert is_known_profile("") is False
308         assert is_known_profile("banana") is False
309
310
311     def test_preset_for_returns_copy():
312         a = preset_for("cpu-mock")
313         a["indexing"]["detector_impl"] = "mutated"
314         b = preset_for("cpu-mock")
315         assert b["indexing"]["detector_impl"] == "stub"  # module table not mutated
316         assert preset_for("") == {}
317         assert preset_for("banana") == {}
318
319
320     def test_save_default_config_includes_deployment(tmp_path):
321         p = save_default_config(str(tmp_path / "out.json"))
322         saved = json.loads(p.read_text())
323         assert saved["deployment"] == {"profile": "", "compute_target": ""}
324         cfg = load_config(str(p))
325         assert cfg.deployment.profile == ""
326
327
328     def test_repo_config_json_has_no_profile():
329         """The shipped config.json must not silently opt into a profile (default path stays
today's
330         behaviour for the real deployment)."""
331         repo_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
332         cfg_path = os.path.join(repo_root, "config.json")
333         if not os.path.exists(cfg_path):
334             pytest.skip("no repo config.json")
335         cfg = load_config(cfg_path)
336         assert cfg.deployment.profile == ""
337
```

8. user (2026-06-21T07:24:03.341Z)

```
1       # Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storage
2
3       > **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** ·
**Owner:** Avi · **Created:** 2026-06-21 · **Last updated:** 2026-06-21
4       > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to
`docs/archives/past_projects/` when DONE)
5       > **Tracking anchors:** §7 progress tracker is the source of truth; pointer in memory
`current-state` + `docs/NEXT_STEPS.md` once work starts.
6       > **Relation to existing docs:** the productionization successor to the archived
`DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from
[PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) §3; the **truly-local** profile is the
L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend
of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
7       > **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed
**compute-decoupled serving** because **truly-local is a co-equal, first-class profile**, not an
afterthought.
8       > **Honesty note:** claims tagged `[verified]` (checked against code by the design
workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.
9
10      ---
11
12      > **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video
? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next
to the source**. Two design promises around it: a user can **switch local?cloud anytime** (D13,
a headline feature), and execution is **never a surprise** ? auto-detect *suggests*, the user
*confirms* (D15).
13
14      ## 0. TL;DR
15      The rally engine is a **pure function of `(input bytes + config) ? outputs`** in three
```

deterministic stages ? **DETECT ? WINDOW ? STITCH** ? that **no deployment profile may branch on**. A **`ComputeBackend` (deployment profile)** decides *where* the core runs (a local process vs a Cloud Run GPU job); a **`StorageBackend`** decides *where bytes come from* (local FS / USB / mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local** (local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload `<2GB` / gdrive / bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus **cpu-mock** for CI. The single most important caveat: detection is **byte-identical only on the same GPU** (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the **parity guarantee is enforced at the WINDOW + STITCH layer**, not detector-CSV bytes.

1. Problem & goal

- **Why now:** DECOUPLED_COMPUTE proved compute *portability* (M0?M2 + M3.5 on a GCP L4) and is archived; its deferred **M3 (serving endpoint + per-video billing) + M4 (scale-to-zero + cost guard)** become this project. The owner needs **both** a hosted service *and* a truly-local mode, with the **core (detect+stitch) unaffected** by which one runs.
- **The two user-facing outcomes** (owner, 2026-06-21):
  - **(a) Hosted service** ? operate on small **local uploads (`<2GB`)** *and* load from **gdrive / a bucket**, spinning up **Cloud Run for both rally segmentation AND stitching** (stitching is compute-intensive ? run it where its cost is accounted for).
  - **(b) Truly-local** ? on a local machine with videos on **local drives/USB + a local GPU**, run inference **and** stitching **completely locally**.
- **"Good" looks like:** the same `video ? rallies ? reel` core gives identical windows + stitched ranges whether it runs on a laptop or Cloud Run; switching is a **config/startup choice**, never a code branch in the core.

2. Decisions locked

#	Decision	Source	Implication
D1	The core is a **pure 3-stage function** (DETECT/WINDOW/STITCH); **no profile branch inside it**.	design `[verified]`	All profile difference lives in config + the two registries.
D2	**Two registries:** `ComputeBackend` (where) selected by `deployment.profile` + `compute_target`; `StorageBackend` (what-bytes) by `input_backend`/`storage.backend`.	ADR-014	Realizes PLATFORM_ARCHITECTURE's `local|hosted|byo_worker`.
D3	**Profiles shipped:** `truly-local`, `cloud-serving`, `cpu-mock` (CI); `byo_worker` reserved/deferred.	ADR-014	Enum/seam reserved so BYO isn't precluded.
D4	In **cloud-serving, STITCH runs on Cloud Run** (co-located with detect) so CPU/wall + egress are metered into the per-job cost ledger.	owner (a)	Stitch-on-laptop would hide real cost.
D5	**Parity enforced at WINDOW+STITCH** (byte-exact for a fixed trajectory); **detect** is byte-exact only same-GPU.	deploy report 2026-06-20	Cross-GPU asserted at the rally/window level, never CSV bytes.
D6	**Default-OFF, smallest-safe-first, one PR per milestone**; any core-caller change (e.g. configurable output base) ships behind a flag + a Launch Report.	[[launch-report-convention]]	Zero-regression discipline.
D7	**Cloud Run cold-start: SCALE-TO-ZERO** ? accept the ~1?3 min cold-start (it's an async job: upload?poll?reel, amortized over the multi-minute job). **No warm pool** (that ~$500/mo idle would break per-video billing).	owner 2026-06-21	M6/M7. Revisit a warm lane only on a UX complaint.
D8	**Pricebook = a versioned DB table**; cost computed in **USD** (matches GCP billing = one source of truth), **displayed in INR** at a configurable FX rate (Bangalore market); re-versioned when GCP prices change.	owner 2026-06-21	M8.
D9	**Edge auth = Cloudflare Access** (reuse the site's existing CF). Lock the Cloud Run **ingress to the CF proxy** + **verify the `Cf-Access` JWT signature** (so a direct hit to the Cloud Run URL can't spoof the identity header).	owner 2026-06-21	M9. One identity system.
D10	**Free-tier "data-for-reels" trade:** free reels in exchange for **training rights** (uploaded footage + derived trajectories/labels); **paid tier = no-train** (privacy is the paid feature). **Carve-outs:** train ONLY on attested **ADULT** footage (minor footage ? reel yes, training pool **NO** ? DPDP/GDPR need verifiable parental consent); train on **DERIVED** data + **auto-delete raw**; **ToS counsel-reviewed**; live training-on-uploads **gated to M9** (public signup). Truly-local needs no DPA.	owner 2026-06-21	M9 + D12; ties [[paper-coauthor-aim]] / PERSONALIZATION flywheel.
D11	**Quality floor: Gemini handoff ON** for cloud-serving until the owned model

clears the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a **Launch Report**; the owned model runs in **shadow/eval** meanwhile. (Gemini = serving/inference ONLY, never a training source ? CLAUDE.md guardrail.) | owner 2026-06-21 | M7 + D12. |
39 | D12 | **Data-flywheel harness is IN SCOPE (owner add, Q5):** capture the (consented, adult) uploaded video + the Gemini reel/rally output ? **reliably store** in the per-video workspace/bucket ? **run shadow evals** (owned-vs-Gemini, dual-eval-set) ? **promote good ones to the golden TRAINING set** (human-reviewed labels) so the owned model climbs toward the D11 floor (then Gemini flips off). | owner 2026-06-21 | new **M11**; ties D10 (consent) + `data_pipeline/` + the eval/experiment harness. |
40 | D13 | **Profile is user-switchable ANYTIME ? a headline FEATURE.** The same user runs **local today, cloud tomorrow, back to local** ? it's just a config/startup choice; the pure core gives **equivalent** output either way. **Advertise it** ("your machine *or* our cloud ? your call, switch anytime"). | owner 2026-06-21 | Honest caveat: equivalent at the **rally/window** level, not bit-identical across GPUs (D5); a single job runs in one profile, switching is *between* jobs. |
41 | D14 | **NORTHSTAR ? operate toward this:** a **mobile, app-like** flow ? point at a **Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next to the source**. | owner 2026-06-21 | Implies a **`gdrive-api` (OAuth) StorageBackend** (read source + write reel back) ? distinct from the local **mount** backend; **resolves §8 #7**, new **M12**. The mobile UI is the khelsutra track; the engine must support it (async jobs + status + Drive round-trip / signed URL). Net-new scope (OAuth + Drive write); not necessarily v1, but the design must not preclude it (the `StorageBackend` ABC accommodates it). |
42 | D15 | **Auto-detect SUGGESTS, the user CONFIRMS ? execution is NEVER a surprise.** A GPU owner may still choose cloud; **cloud (= spend) is never entered silently**; the active profile is always explicit + surfaced. | owner 2026-06-21 | Amends §4.3 (auto-detect = a proposed default, not an auto-decision). |
43
44 ## 3. Foundation ? evolution / ADR lineage ? *trace the idea end-to-end*
45 This project is the latest step in a long decision chain; each ADR contributed a piece and a **subtlety that carries forward**:
46
47 | ADR / doc | Contributed | Subtlety carried into this project |
48 |---|---|---|
49 | **ADR-005** | Permissive licensing (MIT/Apache-2.0/BSD only) | The engine + any served model + the **container image** (ffmpeg, fonts, weights) stay commercial-clean. |
50 | **ADR-008** | Owned-model topology; **train==serve featurizer single-sourcing** | The "core unaffected across profiles" is the same parity discipline, extended from train/serve to **local/cloud**. |
51 | **ADR-009** | KhelSutra Guru; **local-first delivery** | The **truly-local profile** is the direct continuation ? privacy/offline self-host stays first-class. |
52 | **ADR-010** | DECOUPLED: DO?GCP pivot | The cloud-compute origin (GPU + co-located bucket). |
53 | **ADR-011** | DECOUPLED scope = M0?M2+M3.5; **deferred M3 (serving) + M4 (billing/cost-guard)**; overrode the *"rent nothing / not a service"* posture | **This project realizes M3/M4.** Carries §06's owner-gated gates: **DPA/consent for non-owner uploads, collector `md5` keying, WASB-C3**. |
54 | **ADR-012** | GPU-native; TPU deferred; **Cloud Run+GPU scale-to-zero = serving model**; **NVDEC** named as the decode lever | The serving substrate + the 7-step seed scope this expands. |
55 | **ADR-013** | Drop DO; **GCP sole compute stack**; $200 DO credits ? non-compute only | The provider for cloud-serving; DO Spaces remains only a possible S3 *storage* target. |
56 | **? ADR-014 (new)** | **Compute-decoupled serving:** one pure core, profile-selected compute+storage; stitch-where-metered | This doc. **Refines** ADR-011/012, does **not** supersede them. |
57 | PLATFORM_ARCHITECTURE.md | The `ComputeBackend`/`StorageBackend` registries | The abstraction this realizes in code. |
58 | VIDEO_LOCALITY_MODEL.md / DESKTOP_APP_PLAN.md | L0 fully-local tier / `LocalDesktop` backend | The truly-local profile = these, productionized. |
59 | 07-COMPUTE-ABSTRACTION (archived) | `DeviceContext`/`DeviceProfile` (designed) | WASB hardcodes `device='cuda'` with **no CPU fallback** ? a portability gap M4 closes. |
60 | DEPLOY_REPORT_GCP_2026-06-20 (archived) | First real L4 run; **cross-GPU sub-pixel parity finding** | Why D5 (parity at window level, not CSV bytes). |
61
62 ## 4. Design / architecture

63   See [`architecture.svg`](architecture.svg) for the picture. **The core never changes; the box around it does.**
64
65   ### 4.1 The core contract (must stay byte/behaviour-identical) `[verified]`
66   1. **DETECT (GPU):** `DetectorRunner.run_predict(video_path, output_dir, video_id) ? CSV[Frame,Visibility,X,Y]` (`backend/pipeline/detectors/base.py:164`). Three interchangeable impls ? `WasbRunner` (WSL), `NativeWasbRunner` (Linux GPU), `StubDetectorRunner` (CPU mock) ? emit the **identical CSV schema**. The trajectory CSV is the stable artifact boundary.
67   2. **WINDOW (CPU pure fn):** `trajectory_to_action_windows(points, fps, frame_width, ?) ? windows` (`tracknet_runner.py:281`). A shared `@staticmethod`, zero GPU/IO/randomness ? same trajectory + config = byte-identical candidate windows. Used by every runner *and* the eval/regression stack verbatim.
68   3. **STITCH (CPU/ffmpeg):** `VideoCompiler.compile_highlights(raw_video, segments, out_name) ? reel` (`stitcher/compiler.py:303`). Pure FS+subprocess (merge ? ffprobe ? watermark ? per-clip libx264 trim ? concat). Deterministic for a given input + ranges + watermark config.
69
70   **Non-goal (the trap to avoid):** *no* code branch inside detect/window/stitch keyed on profile ? the same discipline the experiment harness already enforces (a disabled cue is byte-identical to CONTROL).
71
72   ### 4.2 The deployment profiles
73 | Profile | Compute | Storage | Orchestration | When |
74 |---|---|---|---|---|
75 | **truly-local** | `compute_target=local_gpu`; `detector_impl=native` (Linux) / `auto` (WSL); stitch **in-process** | `local` (or `gdrive` = mounted Drive) | `python -m backend.main` or `worker infer` CLI; existing async job worker | Self-host / offline / privacy / dev-on-a-GPU-box ? owner (b) |
76 | **cloud-serving** | `compute_target=cloud_run`; `detector_impl=native` (L4, cuda); **detect AND stitch on the same Cloud Run job** | `gcs` (or gdrive mount / assembled upload); per-video workspace; reels via signed URL | Cloud Run control plane enqueues ? Cloud Run GPU job pulls/runs/pushes; `WAIT_AND_RESUME` reschedules the control-plane job; scale-to-zero | Hosted SaaS ? owner (a) |
77 | **cpu-mock** | `compute_target=cpu_mock`; `detector_impl=stub` (synth/replay) | `local`/in-memory fakes | `run_all_tests.py` / pytest | CI, regression, profile-parity, GPU-free dev |
78 | *byo_worker* (deferred) | tenant GPU via outbound pull agent | tenant's own bucket (signed URLs) | long-poll pull; same job table scoped by tenant | Privacy-sensitive enterprise ? **DEFERRED**, enum/seam reserved |
79
80   ### 4.3 Profile selection (the "startup/setup config") `[design]`
81   ONE new top-level `deployment` section on `AppConfig` + preset application + env auto-detect in `load_config`:
82   - `deployment.profile ? {truly-local | cloud-serving | cpu-mock}` (empty = today's manual knobs, **fully backward-compatible**).
83   - Selecting a profile applies a **coherent preset bundle** of *existing* knobs that today drift independently: **INPUT** (new `input_backend`/`input_root`, parallel to the output-side `StorageConfig`), **COMPUTE** (new `compute_target` enum locking `detector_impl` + `skip_ai_handoff` + workspace strategy), **STORAGE** (`storage.backend`/`workspace_root`/`single_folder_per_video`, all already exist).
84   - **Precedence:** built-in defaults ? profile preset ? `config.json` ? env (`RALLY_DETECTOR_IMPL`, `WASB_*`, `DEPLOYMENT_PROFILE`, ?) ? worker `--set k=v`. Every knob stays individually overridable.
85   - **Env auto-detect SUGGESTS, the user CONFIRMS (D15 ? no surprise execution):** the env gives a *proposed* default (`K_SERVICE` ? cloud-serving; `CI=true` ? cpu-mock; else `torch.cuda.is_available()` ? truly-local), but the user **explicitly confirms or overrides** it ? a GPU owner may still pick cloud, and **cloud (= spend) is NEVER entered silently**. The active profile is always surfaced (logged/shown). **Per-profile startup checks fail/warn fast** (cloud-serving without GCS creds/GPU ? loud error before any job).
86   - The worker (`cmd_infer`/`cmd_train`) is **already profile-agnostic** (reads `config.storage`, accepts `--config`/`--set`) ? no worker rewrite.
87
88   ### 4.4 Compute placement ? and *why stitch runs on Cloud Run* `[design]`
89   DETECT on GPU everywhere; WINDOW CPU-pure everywhere; STITCH is real CPU work. In **cloud-serving, stitch is co-located in the Cloud Run job** so `gpu_active_seconds` (detect) + `wall_seconds` (detect+stitch) + `bytes_out` (reel egress) land on **one metered invocation** ?

the per-job cost ledger. Running stitch on a laptop would leave that cost **invisible/unattributed** ? exactly what the owner wants to avoid. (Stitch stays synchronous in the GPU job for the common one-reel case; a queued CPU-only stitch fallback only if compiles get bursty ? the job table + `claim_due_job` already supports both.)

4.5 Reuse map ? most of the seams already exist `[verified]`

Exists: the 3-stage core; `_resolve_runner` (the single compute seam); `StubDetectorRunner` (`replay_csv` = byte-exact $0 anchor); `StorageBackend` ABC + 4 backends + `StorageRef.parse_uri`; the worker CLI (fetch/put, `--config`/`--set`); the async job control plane (`jobs` table, `claim_due_job` CAS); `ExecutionPolicy WAIT_AND_RESUME`/`JobWaiting` (carries to Cloud Run dispatch verbatim); the chunked-upload router (`<2GB`); `skip_ai_handoff` (LocalNoAI); storage fakes; golden fixtures + experiment harness. **Partial:** per-video workspace helper (default-OFF); `DeviceContext` (designed); edge-auth (v4 `user_id` columns exist). **Net-new:** the `deployment` section; `input_backend`; configurable output base (kill hardcoded `output/`); Cloud Run dispatch shim + container image; cost ledger + pricebook; edge-auth header extraction; startup env detection.

4.6 The data-flywheel harness (D12 / M11) `[design]`

Gemini-on (D11) ships good reels now; this harness is **how we earn the right to flip Gemini off**. On a **consented, adult** cloud-serving job (the D10 trade), the pipeline additionally:

1. **Captures** the source upload + the Gemini reel + the rally output (intervals/report) into the per-video workspace (`workspaces/<video_id>/`), instead of the no-consent auto-delete path.
2. **Reliably stores** them durably in the bucket (the consented-retention bucket; raw video kept only for the training pool, not the no-consent path).
3. **Runs shadow evals** ? the owned model scores the same video alongside Gemini (`backend/eval/experiment.compare` + the dual-eval-set, [[eval-dual-set-regression]]) ? tracks the owned-vs-Gemini gap per job (the live read on "is the floor reachable yet?").
4. **Promotes to golden** ? captured videos that pass review become **golden TRAINING rows** (human-reviewed labels via the annotation flow ? `data_pipeline/`), growing the corpus ? the owned model climbs to the D11 floor ? flip Gemini off via a Launch Report.

This is the consent?capture?eval?goldenize **flywheel** (the cloud realization of the PERSONALIZATION contribution model): every consented free reel makes the owned model better. Reuses `data_pipeline/`, `backend/eval/`, and the M9 consent gate; **adults-only**, raw-retention gated by the D10 consent.

6. Honest limits ? what this does NOT do / prove

- A green CI suite proves **plumbing + determinism (window+stitch) + profile-parity** for $0 ? it does **NOT** prove field quality, nor that real-GPU detection is good (that's owner-side, real-video).
- **Cross-GPU detection is NOT byte-identical** ? parity is a window-level claim, not a CSV-byte claim.
- The cloud `0.1.0-l4` weights are **n=2 plumbing-grade** ? cloud-serving ships with **Gemini handoff ON** until the owned model clears a floor (open Q).
- This does not resolve **WASB-C3** (sharing a trained model) or the **DPA/consent** gate ? both remain owner/counsel-gated before public, non-owner serving.

7. Deliverables & progress tracker ? **source of truth**

Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per row**, off `origin/master`, no stacking. Default-OFF where it touches core callers.

ID	Deliverable	Depends	Gated?	Status	PR
M0	This design doc + project dir + `runlogs/` + ADR-014	?	owner sign-off	?	**? APPROVED 2026-06-21**
M1	`deployment.profile` + `compute_target` config + preset bundles + precedence/auto-detect; unit tests (no behaviour change, profile='' default)	M0	safe	?	?
M2	`input_backend`/`input_root`; wire main CLI/API input through `StorageRef.parse_uri` (local=identity); storage-fake tests	M1	safe	?	?
M3	Configurable output/scratch base ? kill hardcoded `output/` (`trajectory_hybrid:237,313`, `yolo_hybrid:407`, `gemini`); **DEFAULT stays `output/`**; golden + stub-E2E byte-identical	M1	? default-OFF + Launch Report on flip	?	?
M4	`DeviceContext` + WASB **CPU-fallback**; unified healthcheck/device wiring	M1

safe (cuda default unchanged) | ? | ? |
118 | M5 | **truly-local profile** end-to-end (preset + startup checks); first committed **runlog** (truly-local sample run) | M2,M3,M4 | owner real-GPU | ? | ? |
119 | M6 | Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) + **containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner (GCP spend) | ? | ? |
120 | M7 | **cloud-serving profile** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch on Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet) | M6 | ? owner (spend) | ? | ? |
121 | M8 | Per-job **cost ledger** (v_ migration: `owner_id, gpu_active_seconds, wall_seconds, bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation + `/api/.../cost` | M7 | ? owner (billing) | ? | ? |
122 | M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`; DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? | ? |
123 | M10 | `byo_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner approval | ? | ? |
124 | **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent gate (M9). | M7,M9 | ? owner (consent/privacy) | ? | ? |
125 | **M12** | **`gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the source from + writes the stitched reel **back to the user's Google Drive next to the source** (per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |
126
127   **Testing strategy is a first-class deliverable** ? [`TESTING_STRATEGY.md`](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ? [`runlogs/`](runlogs/).
128
129   ## 8. Open questions ? owner *(blocking-gates vs nice-to-knows)*
130   **? The 5 gating questions are LOCKED (owner, 2026-06-21) ? see §2 D7?D12:**
131   1. ? **Cold-start ? D7** (scale-to-zero, accept cold-start).
132   3. ? **Pricebook ? D8** (versioned DB table; USD internal / INR display).
133   4. ? **Edge auth ? D9** (Cloudflare Access; verify Cf-Access JWT, lock ingress to proxy).
134   5. ? **DPA/consent ? D10** (free-tier data-for-reels trade; adults-only; derived-data; counsel-gated to M9).
135   6. ? **Quality floor ? D11** + the **data-flywheel harness D12 / M11** (Gemini-ON until the owned model clears the floor; capture?store?eval?goldenize meanwhile).
136
137   **Still open (non-gating, smaller):**
138   2. **Stitch placement:** always co-located in the detect job (simplest, one metered unit) vs split to a cheap CPU Cloud Run service when bursty? *Default co-located unless metrics say otherwise.*
139   7. ? **RESOLVED ? D14:** cloud needs a **`gdrive-api` (OAuth) StorageBackend** (read the source from + write the reel back to the user's Drive) ? the Northstar's Drive round-trip ? distinct from the local **mount** backend (which stays for truly-local). New milestone **M12**.
140   8. **NVDEC/NVENC** in the Cloud Run image (cut stitch wall-time/cost) vs libx264 (determinism/parity)?
141   9. **Input cap:** is `<2GB` a hard client reject, or accept to the 4GB upload cap and route larger files elsewhere?
142   10. **Directory name:** `COMPUTE_DECOUPLED_SERVING/` (chosen) vs keep `CLOUD_RUN_GPU_SERVING/` ? confirm.
143
144   ## 9. Definition of done
145   - [ ] M1?M4 land (config + input + output-base + device layer), each green on golden cross-OS + experiment no-regression.
146   - [ ] **Profile-parity test green** ? truly-local vs cloud-mock produce byte-identical windows + segment ranges.
147   - [ ] **truly-local** (M5) and **cloud-serving** (M7) each have a committed **runlog / DEPLOY_REPORT** with a real sample run.
148   - [ ] Cost ledger (M8) makes GPU + **stitch** + egress cost visible per job; edge auth (M9) gates multi-tenant.

```
149     - [ ] **Data-flywheel harness (M11)** captures?stores?shadow-evals?goldenizes consented
adult jobs; the owned model is on a measured path to the D11 floor.
150     - [ ] ? flip Status DONE, archive to `docs/archives/past_projects/`.
151
152     ## 10. References
153     **ADR lineage:** ADR-005/008/009/010/011/012/013 ? **ADR-014**
(`docs/archives/decisions/DECISIONS.md`). **Strategy:** `PLATFORM_ARCHITECTURE.md`,
`HOSTING_PLAN.md`, `VIDEO_LOCALITY_MODEL.md`, `DESKTOP_APP_PLAN.md`,
`LOCAL_APPLIANCE_ARCHITECTURE.md` (khelsutra). **Archived predecessor:**
`docs/archives/past_projects/DECOUPLED_COMPUTE/` (esp. `05-COST-ATTRIBUTION.md`, `07-COMPUTE-
ABSTRACTION.md`, `06-RISKS-AND-OPEN-QUESTIONS.md`, `DEPLOY_REPORT_GCP_2026-06-20.md`). **Code
anchors `[verified]`:** `backend/pipeline/detectors/base.py:164`, `tracknet_runner.py:281`,
`stitcher/compiler.py:303`, `segmenters/trajectory_hybrid.py:65,237`, `backend/storage/*`,
`backend/worker/__main__.py:88`, `backend/api/{uploads,job_worker}.py`,
`backend/infrastructure/database.py`. **Design provenance:** workflow `wf_56274ff0` (4 code-
mappers ? synthesis; 58 findings).
154
155     ---
156     ### Changelog
157     - 2026-06-21 ? created from the design workflow output; M0 in progress.
158
```

9. user (2026-06-21T07:24:03.520Z)

```
1       """Basic tests for the typed configuration package.
2
3       These ensure:
4       - Defaults are sensible and come from the models.
5       - File overrides work.
6       - Old config files with extra keys are tolerated.
7       - save_default_config produces loadable output.
8       """
9       import json
10      import tempfile
11      import os
12
13
14      from backend.config import load_config, save_default_config, AppConfig
15
16
17      def test_load_default_config():
18          # Force load without the project's config.json so we test the model built-in
defaults
19          cfg = load_config("/nonexistent/path/that/does/not/exist/12345/config.json")
20          assert isinstance(cfg, AppConfig)
21          assert cfg.sport == "badminton"
22          assert cfg.indexing.default_segmenter == "yolo_hybrid"
23          assert cfg.validation.min_blur_threshold == 20.0  # updated default from review
24          assert cfg.indexing.ai_max_workers == 5
25
26
27      def test_logging_config_default_quiets_http():
28          """verbose_http defaults to False so run.log is quiet unless explicitly turned
up."""
29          cfg = load_config("/nonexistent/path/12345/config.json")
30          assert cfg.logging.verbose_http is False
31
32
33      def test_skip_ai_handoff_default_off():
34          """Fully-local mode is opt-in: skip_ai_handoff defaults False so the AI handoff
runs."""
35          cfg = load_config("/nonexistent/path/12345/config.json")
36          assert cfg.indexing.skip_ai_handoff is False
37
38
```

```
    def test_skip_ai_handoff_file_override():
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "skip.json")
            with open(p, "w") as fh:
                json.dump({"indexing": {"skip_ai_handoff": True}}, fh)
            cfg = load_config(p)
            assert cfg.indexing.skip_ai_handoff is True


    def test_logging_config_file_override():
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "logging.json")
            with open(p, "w") as fh:
                json.dump({"logging": {"verbose_http": True}}, fh)
            cfg = load_config(p)
            assert cfg.logging.verbose_http is True


    def test_fusion_config_defaults_reproduce_control():
        """The fusion block defaults must be no-ops: wasb substrate, no cues, no gating, no
mask.

        Gate A (min_crossings) defaults to 0 = OFF: the #146 served-default flip was
REVERTED
        (eval?serve ? the +0.074 was on the harness's recall-oriented PROPOSAL windowing; on
the
        served COARSE windowing Gate A is neutral/negative, ?0.008, dropping real rallies).
See
        docs/QUALITY_ITERATIONS.md and the served litmus in
tests/test_served_gate_a_regression.py.
        """
        cfg = load_config("/nonexistent/path/12345/config.json")
        f = cfg.indexing.fusion
        assert f.substrate == "wasb"
        assert f.cue_flags == {}            # person cue OFF
        assert f.min_crossings == 0         # Gate A OFF (default) ? #146 served flip
reverted
        assert f.min_players == 0           # Gate B OFF
        assert f.court_polygon is None      # no mask
        assert f.velocity_threshold == 0.02


    def test_fusion_config_gate_a_opt_in_via_min_crossings():
        """Gate A is opt-in: setting min_crossings explicitly enables it (the eval-only
operating
        point), while the served default stays OFF (#146 flip reverted)."""
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "optin.json")
            with open(p, "w") as fh:
                json.dump({"indexing": {"fusion": {"min_crossings": 2}}}, fh)
            cfg = load_config(p)
            assert cfg.indexing.fusion.min_crossings == 2   # Gate A ON only when explicitly
set


    def test_served_default_segmenter_is_gemini():
        """The shipped config.json served default is the prior detector (gemini) ? the #146
flip
        to fusion_hybrid was REVERTED as a confirmed served regression (eval?serve)."""
        repo_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
        cfg = load_config(os.path.join(repo_root, "config.json"))
        assert cfg.indexing.default_segmenter == "gemini"
        # ...and the served fusion default keeps Gate A OFF (no served gating).
        assert cfg.indexing.fusion.min_crossings == 0

```

```

    def test_fusion_config_file_override():
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "fusion.json")
            with open(p, "w") as fh:
                json.dump({"indexing": {"fusion": {
                    "substrate": "tracknet", "min_crossings": 2,
                    "cue_flags": {"person": True}, "court_polygon": [[0.1, 0.1], [0.9, 0.1],
[0.5, 0.9]],
                }}}, fh)
            cfg = load_config(p)
            f = cfg.indexing.fusion
            assert f.substrate == "tracknet"
            assert f.min_crossings == 2
            assert f.cue_flags == {"person": True}
            assert len(f.court_polygon) == 3
            # An old config without a fusion block still loads (defaults applied).
            assert cfg.indexing.wasb.velocity_threshold == 0.02


    def test_file_override():
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "override.json")
            data = {
                "indexing": {
                    "default_segmenter": "gemini",
                    "ai_max_workers": 3,
                },
                "validation": {
                    "min_blur_threshold": 42.0,
                },
            }
            with open(p, "w") as f:
                json.dump(data, f)

            cfg = load_config(p)
            assert cfg.indexing.default_segmenter == "gemini"
            assert cfg.indexing.ai_max_workers == 3
            assert cfg.validation.min_blur_threshold == 42.0
            # Unspecified keys fall back to defaults
            assert cfg.sport == "badminton"


    def test_extra_keys_tolerated():
        """Old or future keys in config.json must not break loading."""
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "with_extra.json")
            data = {
                "sport": "badminton",
                "indexing": {"default_segmenter": "yolo_hybrid"},
                "some_future_key": "ignored_for_now",
                "validation": {"relevance": {"court_pixels_min_ratio": 0.1}},
            }
            with open(p, "w") as f:
                json.dump(data, f)

            cfg = load_config(p)
            assert cfg.indexing.default_segmenter == "yolo_hybrid"
            assert cfg.validation.relevance.court_pixels_min_ratio == 0.1


    def test_unknown_keys_warn_with_dotted_paths(tmp_path, caplog):
        """Typo'd keys must be LOUD: top-level extras are kept-but-unread and unknown
        section keys are silently dropped, so without this warning a typo just makes
        the knob a no-op (the audit's 'silently dropped config' class)."""
```

```
        p = tmp_path / "typo.json"
        p.write_text(json.dumps({
            "some_future_key": 1,                                  # top-level extra
            "indexing": {"velocty_threshold": 0.5},                # typo'd section key
            "validation": {"relevance": {"unknown_gate": True}},   # doubly nested
        }))
        import logging
        with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
            cfg = load_config(str(p))
        assert isinstance(cfg, AppConfig)  # still non-fatal
        joined = " ".join(r.message for r in caplog.records)
        assert "some_future_key" in joined
        assert "indexing.velocty_threshold" in joined
        assert "validation.relevance.unknown_gate" in joined


    def test_recognized_config_does_not_warn(tmp_path, caplog):
        p = tmp_path / "clean.json"
        p.write_text(json.dumps({
            "sport": "badminton",
            "indexing": {"default_segmenter": "gemini", "wasb": {"velocity_threshold":
0.02}},
        }))
        import logging
        with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
            load_config(str(p))
        assert not [r for r in caplog.records if "CONFIG WARNING" in r.message]


    def test_non_object_config_degrades_to_defaults(tmp_path, caplog):
        """A config.json that is valid JSON but NOT an object (`[]`, `[1]`, `"x"`, `42`,
`true`,
        `null`) must degrade to a fully-defaulted AppConfig + a warning ? never crash
startup.
        Regression: previously a truthy non-mapping hit `.items()` (AttributeError) and a
falsy one
        broke the `{**defaults, **file_data}` unpack (TypeError), turning an operator typo
into a hard
        server/CLI startup crash."""
        import logging
        default_dump = AppConfig().model_dump(mode="json")
        for raw in ("[]", "[1, 2]", '"x"', "42", "true", "null"):
            p = tmp_path / "bad.json"
            p.write_text(raw)
            caplog.clear()
            with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
                cfg = load_config(str(p))              # must not raise
            assert isinstance(cfg, AppConfig)
            assert cfg.model_dump(mode="json") == default_dump, f"{raw} should yield pure
defaults"
            assert any("not a JSON object" in r.message for r in caplog.records), \
                f"{raw} should log the non-object warning"


    def test_save_default_config_roundtrip():
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "saved.json")
            saved = save_default_config(p)
            assert str(saved) == p
            assert os.path.exists(p)

            cfg = load_config(p)
            assert cfg.indexing.default_segmenter == "yolo_hybrid"
            assert cfg.validation.min_blur_threshold == 20.0

```

```
218
219     def test_model_is_the_source_of_truth():
220         """Changing a default in the model should be reflected without touching json."""
221         # This is a documentation test more than anything.
222         cfg = AppConfig()
223         assert cfg.indexing.tracknet.velocity_threshold == 0.02
224
225
226     def test_output_dir_field_default_and_override():
227         """output_dir now lives on OutputConfig (was a write-only runtime hack), so
228         /api/compile and the CLI resolve the same value. Default + file override + the
229         runtime mutation that backend/main.py::main performs for --output-dir."""
230         cfg = AppConfig()
231         assert cfg.output.output_dir == "output"  # built-in default
232
233         # The CLI override path: main() assigns args.output_dir onto the typed model.
234         cfg.output.output_dir = "/tmp/custom_out"
235         assert cfg.output.output_dir == "/tmp/custom_out"
236
237         # File override is honored on load.
238         with tempfile.TemporaryDirectory() as td:
239             p = os.path.join(td, "out_override.json")
240             with open(p, "w") as f:
241                 json.dump({"output": {"output_dir": "reels", "db_name": "x.db"}}, f)
242             loaded = load_config(p)
243             assert loaded.output.output_dir == "reels"
244             assert loaded.output.db_name == "x.db"
245
246     def test_config_shallow_section_replacement():
247         """Positive test for Item 2 (config loader): shallow section replacement.
248
249         File-provided 'indexing' section replaces the defaults 'indexing' section entirely
250         (no deep merge of sub-keys), per loader's {**defaults, **file_data}.
251         """
252         with tempfile.TemporaryDirectory() as td:
253             p = os.path.join(td, "shallow.json")
254             with open(p, "w") as fh:
255                 json.dump({"indexing": {"default_segmenter": "motion"}}, fh)
256             cfg = load_config(p)
257             assert cfg.indexing.default_segmenter == "motion"
258             # Pydantic defaults fill the rest of the (replaced) section
259             assert cfg.indexing.ai_max_workers == 5
260
261
262     def test_config_unknown_key_paths_warning_and_tolerate():
263         """Positive test for Item 2: _unknown_key_paths warning for unknown top-level keys,
264         but tolerated (extra='allow'), and unknown section keys dropped silently.
265         """
266         with tempfile.TemporaryDirectory() as td:
267             p = os.path.join(td, "unknown.json")
268             with open(p, "w") as fh:
269                 json.dump({
270                     "indexing": {"default_segmenter": "motion"},
271                     "completely_unknown_top": "bar",
272                     "validation": {"unknown_in_section": 99}  # dropped
273                 }, fh)
274             cfg = load_config(p)
275             assert cfg.indexing.default_segmenter == "motion"
276             # tolerated, no crash
277
278
279     def test_get_returns_dict_for_nested_sections_legacy_compat():
280         """Regression: legacy consumers (validators/segmenters) do
281         ``config.get("validation", {}).get("min_resolution_width", ...)``. The dict-compat
282         .get must return nested SECTIONS as plain dicts (not Pydantic models), or that
```

```
283         chained access raises 'X object has no attribute get' on every real run.
284
285         This is the original strong legacy test (restored per #160 review). It covers the
286         full set of footgun guards: dict sections, nested dicts, scalar leaves with
defaults,
287         missing key fallbacks, and leaf keys discoverable inside sections.
288         """
289         cfg = load_config("/nonexistent/123/config.json")
290         validation = cfg.get("validation", {})
291         assert isinstance(validation, dict)
292         assert validation.get("min_resolution_width") == 1280
293         # nested-of-nested is also dict-accessible
294         assert isinstance(validation.get("relevance"), dict)
295         assert validation["relevance"].get("court_pixels_min_ratio") == 0.05
296
297         # scalar leaves are returned unchanged; a present scalar ignores the default arg;
298         # a missing key falls back to the default
299         assert cfg.get("ai_provider") == "gemini"
300         assert cfg.get("ai_provider", "ignored-default") == "gemini"
301         assert cfg.get("does_not_exist", "fallback") == "fallback"
302
303         # leaf keys inside any section resolve via the dynamic section scan (not a hardcoded
304         # list), so a future top-level section's keys would be discoverable too
305         assert cfg.get("min_resolution_width") == 1280  # lives inside `validation`
306         assert cfg.get("begin_padding") == 0.5          # lives inside `stitching`
307
308         # legacy chained access still works
309         assert cfg.get("indexing", {}).get("default_segmenter") == "yolo_hybrid"
310
311
312     def test_config_roundtrip_and_typed_reaches_pipeline(monkeypatch):
313         """Positive for Item 2: save_default_config + load_config round-trips;
314         typed AppConfig is what reaches pipeline code (e.g. main uses global_config as
AppConfig).
315         """
316         with tempfile.TemporaryDirectory() as td:
317             p = os.path.join(td, "roundtrip.json")
318             save_default_config(p)
319             cfg = load_config(p)
320             assert isinstance(cfg, AppConfig)
321             assert cfg.sport == "badminton"
322
323         # positive that typed is used in pipeline code (global_config is set at startup in
main();
324         # monkeypatch here so the test does not depend on server initialization)
325         import backend.main as main_module
326         from backend.main import get_config
327         monkeypatch.setattr(main_module, "global_config", cfg)
328         assert isinstance(get_config(), AppConfig)
329
```

10. assistant (2026-06-21T07:24:07.115Z)

Now let me trace the precedence carefully. Let me look at one subtle area ? the env-wins re-stamp path and `compute_target` consistency. Let me also check the `model_dump` behavior of the deployment section when a profile re-stamps the profile but not compute_target.

Let me probe the key precedence scenarios by running actual code paths.

11. user (2026-06-21T07:24:13.007Z)

```
profile= cpu-mock
compute_target= cpu_mock
detector_impl= stub
storage.backend= local
```

```
skip_ai_handoff= True
```

12. assistant (2026-06-21T07:24:20.964Z)

```
Interesting. The env-wins case works correctly because `file_data` only set `profile`, not
`compute_target`, so the preset's `cpu_mock` flows through. But let me probe the dangerous case:
env picks a profile WHILE config.json explicitly sets `compute_target`. That's the precedence
interaction that could go stale.
```

13. user (2026-06-21T07:24:24.920Z)

```
[CONFIG] deployment profile 'cpu-mock' normally uses compute_target='cpu_mock' but config.json
overrides it to 'cloud_run' ? honoring the explicit override.
profile= cpu-mock
compute_target= cloud_run
```

14. assistant (2026-06-21T07:24:29.259Z)

```
This is an honest, warned outcome ? config.json's explicit `compute_target` wins (correct by
precedence), and the mismatch is warned. This is consistent with D15/the documented precedence.
Let me now probe the deep-merge into free-form dict fields (storage.local/s3/gcs/gdrive,
fusion.cue_flags) ? case (d).
```

15. user (2026-06-21T07:24:34.509Z)

```
gcs= {'bucket': 'mybucket', 'region': 'us'}
cue_flags= {'person': True}
storage.backend= gcs
```

16. assistant (2026-06-21T07:24:38.668Z)

```
Now the key question for case (d): could merging-not-replacing a *populated* dict surprise a
user? Since presets never populate `storage.gcs`/`fusion.cue_flags`, the default is `{}`, and
`_deep_merge({}, {file})` == `{file}`. So no surprise there. But there's a subtler asymmetry:
the **no-profile path** uses shallow merge (whole `indexing` section replaced by Pydantic re-
defaulting), while the **profile path** uses deep merge. Let me verify this difference is real
and whether it could matter for a free-form dict where defaults are non-empty. Let me check if
any default dict is non-empty (e.g., validation lists).
```

17. user (2026-06-21T07:24:42.068Z)


```
{
  "validation.relevance.court_green_lower": [
    35,
    40,
    40
  ],
  "security.allowed_upload_extensions": [
    "mp4",
    "mov"
  ],
  "storage.local": {},
  "fusion.cue_flags": {}
}
```


18. assistant (2026-06-21T07:24:47.691Z)

```
The free-form `dict[str, Any]` fields (`storage.local/s3/gcs/gdrive`, `fusion.cue_flags`) all
default to `{}`, so `_deep_merge` into them equals replacement ? no surprise. The non-empty
defaults (`court_green_lower`, `allowed_upload_extensions`) are **lists**, and `_deep_merge`
treats lists as scalars (replace, not merge), so they behave identically to the shallow path.
Good.

Now let me probe the most important asymmetry ? case (e): is the no-profile path EXACTLY the
```

original `{**defaults, **file_data}`? And critically, does the **profile path** deep-merge create a *different* result than the no-profile path would for the same file, in a way that's a behavior change vs pre-M1? Let me verify the deep-merge vs shallow-merge difference is intended and confined to the profile path.

19. user (2026-06-21T07:24:55.907Z)

no-profile path == old shallow path: True
new detector_impl= auto (default auto, file replaced section)
new default_segmenter= motion

20. assistant (2026-06-21T07:25:04.524Z)

The no-profile path is byte-identical to the original `{**defaults, **file_data}`. Now the critical subtlety in case (e): the deep-merge in `_deep_merge(defaults, file_data)` vs the shallow `{**defaults, **file_data}` differ when a profile is active. With a profile, `config.json`'s `indexing` section is **deep-merged** onto `defaults+preset`, so sibling keys *survive*. Without a profile, the `indexing` section **fully replaces** defaults and Pydantic re-defaults siblings. This is documented and intended (the deep-merge is needed for the preset to survive).

But there's a behavioral question: a user who today (pre-M1, no profile) relies on the *shallow-replace* semantics ? e.g., sets `indexing: {default_segmenter: "motion"}` and expects all other `indexing` keys at their built-in defaults ? gets the same result either way because deep-merge of `defaults` and a partial section also yields defaults for unspecified keys. The difference only manifests if preset + file both touch the same section. Let me confirm there's no observable change for a no-profile user (already confirmed: identical).

Now let me check one more precedence subtlety ? env var `RALLY_DETECTOR_IMPL` / `WASB_*`. The spec says precedence is `... < config.json < env`. The loader applies `config.json` as the top layer in the merge but does NOT apply `RALLY_DETECTOR_IMPL`/`WASB_*` env vars in `load_config`. Let me verify where those env vars are consumed.

21. user (2026-06-21T07:25:05.225Z)

Found 16 files
backend\config\models.py
docs\COMPUTE_DECOUPLED_SERVING\README.md
docs\SETUP_NEW_MACHINE.md
docs\COMPUTE_DECOUPLED_SERVING\TESTING_STRATEGY.md
tests\test_worker.py
backend\worker__main__.py
deploy\gcp\README.md
docs\archives\past_projects\DECOUPLED_COMPUTE\HANDOFF.md
deploy\digitalocean\README.md
tests\test_native_wasb_runner.py
backend\pipeline\detectors\native_wasb_runner.py
artifacts\so.json
artifacts\so.py
backend\pipeline\segmenters\trajectory_hybrid.py
tests\test_stub_runner.py
backend\pipeline\detectors\stub_runner.py

22. user (2026-06-21T07:25:08.260Z)

65-   def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
66-   """Resolve the detector runner, honouring the CPU-stub and Linux-native overrides.
67-
68:        ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or ``indexing.detector_impl``:
69-   ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole pipeline
70-   is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"`` swaps
71-   in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
--

```
73-        (the WSL runner) so the default production path is unchanged
74-        (``docs/DECOUPLED_COMPUTE/``).
75-        """
76:        impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
77-        if impl == "stub":
78-            from backend.pipeline.detectors.stub_runner import StubDetectorRunner, StubConfig
79-            logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
--
187-         fps, frame_width = self._probe_fps_width(video_path)
188-
189-         # --- Runner config + windowing thresholds ---
190:         # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
191-         # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
192-         # it delegates to the subclass _build_runner (the real WSL/native runner).
193-         try:
```

23. assistant (2026-06-21T07:25:19.365Z)

This confirms the env vars `RALLY_DETECTOR_IMPL`/`WASB_*` are consumed at **downstream consumption sites** (`_resolve_runner`, native runner), not in `load_config`. The loader's job is `defaults < preset < config.json`; the `< env(per-knob)` layer is applied at the consumer. This matches the design note in loader.py:131-134 ("< env/--set, applied at consumption sites downstream"). So `DEPLOYMENT_PROFILE` is the only env var the loader itself reads, which is correct.

This raises a precedence subtlety worth verifying: the spec's precedence is `preset < config.json < env`. If a profile preset sets `detector_impl=stub` (cpu-mock), and the user *also* sets `RALLY_DETECTOR_IMPL=native` in env, the env must win at the consumption site. Since `_resolve_runner` reads `os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl")`, env does win over the preset-stamped `detector_impl`. Consistent.

Now let me verify case (b) more deeply ? the env-wins re-stamp honesty when config.json sets a profile but no compute_target, and env picks a *different known* profile. The re-stamp sets `profile` to env's value, and `compute_target` comes from... let me trace exactly. In the env-wins case, `preset_for(env_profile)` is deep-merged first (sets compute_target to env profile's canonical), then `file_data` is merged on top. `file_data` has `deployment.profile=cloud-serving` but the loader re-stamps `merged["deployment"]["profile"] = profile` (env's). So profile is honest. compute_target = env preset's canonical (file didn't set it). Already verified above (cpu_mock).

Let me now check a tricky honesty edge: what if config.json's `deployment` value is `None` (JSON null)?

24. user (2026-06-21T07:25:23.179Z)

```
RAISED: ValidationError 1 validation error for AppConfig
deployment
  Input should be a valid dictionary or instance of DeploymentConfig [type=model_type,
input_value=None, input_type=NoneType]
    For further information visit https://errors.pydantic.dev/2.13/v/model_type
```

25. assistant (2026-06-21T07:25:30.232Z)

Let me trace this `deployment: null` case carefully. With env profile set, `profile` is truthy. The merge is `_deep_merge(_deep_merge(defaults, preset), file_data)`. `file_data` has `deployment: None`. In `_deep_merge`, `val=None` is not a dict, so `out["deployment"] = None` ? it overwrites the preset's deployment dict with `None`. Then `merged.setdefault("deployment", {})` ? the key exists (as `None`), so setdefault does nothing. Then `if isinstance(merged["deployment"], dict)` ? it's `None`, so the re-stamp block is **skipped entirely**. So `merged["deployment"]` stays `None`, and `model_validate` raises ValidationError.

Let me check: is this a regression vs pre-M1, or did `deployment: null` always fail? Pre-M1 there was no `deployment` field, so `deployment: null` would've been an extra key (tolerated by

`extra="allow"`). But that's not a real concern ? `deployment: null` is a nonsensical user input, and it fails loudly. However, the bigger question: does this contradict the loader's "bad input is non-fatal" contract that the codebase explicitly upholds elsewhere (non-object config ? degrade + warn; unknown profile ? warn)? Let me check the no-env case (just config.json deployment=null, no profile).

26. user (2026-06-21T07:25:33.553Z)

RAISED: ValidationError 1 validation error for AppConfig

27. assistant (2026-06-21T07:25:38.612Z)

So `deployment: null` raises regardless of profile. This is the same class as `sport: null` or `indexing: null` raising ? Pydantic validates a null section as invalid. That's pre-existing Pydantic behavior for any section field, not M1-specific, and consistent (a null *section* is a hard error, while a null *whole file* degrades). I'll note this is not a new regression ? any `<section>: null` already raised pre-M1 if that section existed. Since `deployment` is new, `deployment: null` is newly reachable as a hard error, but it's nonsensical input and consistent with sibling sections. Low severity at most. Let me confirm a sibling section behaves the same pre-existingly.

28. user (2026-06-21T07:25:40.955Z)

RAISED: ValidationError

29. assistant (2026-06-21T07:25:48.859Z)

Confirmed ? `indexing: null` already raised pre-M1. So `deployment: null` behaving the same is consistent, not a new bug class. Not a finding.

Now let me probe a genuinely interesting precedence interaction in case (b): the `compute_target` mismatch warning. The warning only fires when `ct and canonical and ct != canonical`. Consider: env picks profile `cpu-mock`, but config.json explicitly sets `compute_target: "cpu_mock"` AND profile `cloud-serving`. The env profile is cpu-mock, canonical=cpu_mock, ct from file=cpu_mock ? they match, no warning. But the config.json *profile* said cloud-serving while setting compute_target=cpu_mock... that internal file inconsistency is silently resolved. That's fine ? env won the profile, and the file's compute_target happens to match. No issue.

Now a more subtle one for case (c): the warning path uses `merged["deployment"].get("compute_target")`. After deep-merge, where does `compute_target` come from when config.json sets it? Let me trace the case where config.json sets compute_target but the profile comes from env (so file profile differs). Already covered. Let me check the case the test misses: **config.json overrides compute_target to the EMPTY string** `""`.

30. user (2026-06-21T07:25:52.425Z)

profile= 'cloud-serving'
compute_target= ''

31. assistant (2026-06-21T07:26:01.765Z)

Here's a real subtlety. With `profile: "cloud-serving"` selected and `compute_target: ""` explicitly in config.json:
- The preset sets `compute_target: "cloud_run"`.
- `_deep_merge(...preset..., file_data)` overlays the file's `compute_target: ""`, replacing `cloud_run` with `""`.
- In the re-stamp block, `ct = merged["deployment"].get("compute_target")` ? `""` (falsy).
- The mismatch warning is `if ct and canonical and ct != canonical` ? `ct` is falsy ? **no warning**.
- Result: an active `cloud-serving` profile with `compute_target=""` ? silently inconsistent.

Is this a problem? In M1, `compute_target` is **inert** ("recorded for observability; consumed by the ComputeBackend seam from M2+", models.py:338-339). So a blank compute_target under an

active cloud-serving profile is recorded dishonestly but has zero behavioral effect in M1. It's a latent honesty gap: the profile is `cloud-serving` (which advertises cloud spend) but `compute_target` is blanked silently with no warning. By precedence rules this is "honoring the explicit override," consistent with the `cloud_run`-override case which *does* warn. The asymmetry: overriding to a *wrong non-empty* target warns, but overriding to *empty* (arguably more surprising ? the observability field is now blank) does not. This is a minor observability/honesty gap, not a correctness bug, and compute_target is inert in M1. I'll note it as a low-confidence minor finding.

Let me run the full test suite for these config files to confirm green, then check lint/type per the gate.

32. user (2026-06-21T07:26:11.059Z)

```
..................................................                       [100%]
52 passed in 6.21s
```

33. assistant (2026-06-21T07:26:15.403Z)

Both config test suites pass. Now let me probe one more precedence edge that the spec stresses ? sub-key override with sibling survival when the **profile sets a section the file ALSO touches with a deeply-nested dict**. Specifically `storage` ? preset sets `storage.backend`, file sets `storage.gcs` (sibling). And a tricky one: does an unknown env profile path correctly fall back to the **shallow** no-profile merge (not deep merge)?

34. user (2026-06-21T07:26:20.023Z)

```
[CONFIG] unrecognized deployment profile 'banana' (valid: truly-local, cloud-serving, cpu-mock);
applying no preset.
unknown-env-profile path == shallow pre-M1: True
```

35. assistant (2026-06-21T07:26:25.746Z)

Unknown env profile correctly resets to `""` and takes the shallow no-profile path ? byte-identical to pre-M1.

Let me now examine one final subtle scenario for case (a) ? the spec explicitly asks: does a config.json sub-key override a preset sub-key while a SIBLING preset sub-key survives. The test `test_config_json_overrides_preset_subkey` covers `indexing.detector_impl` override with `skip_ai_handoff` sibling survival. But let me verify cross-section survival and that the **preset's `deployment.compute_target` survives** when file overrides only `deployment.profile`'s sibling. Actually let me probe the inverse: file sets `deployment.compute_target` only (no profile in file), profile comes from env.

36. user (2026-06-21T07:26:29.454Z)

```
[CONFIG] deployment profile 'truly-local' normally uses compute_target='local_gpu' but
config.json overrides it to 'cloud_run' ? honoring the explicit override.
profile= truly-local compute_target= cloud_run
```

37. assistant (2026-06-21T07:26:33.101Z)

Correct and warned. The deep-merge preserved the env-selected profile (re-stamped) while honoring the file's explicit compute_target override, and the mismatch is loudly warned. Precedence holds.

Let me run the broader config-touching test files to be sure nothing else regressed, and a quick ruff check on the changed files.

38. user (2026-06-21T07:26:38.146Z)

```
All checks passed!
---mypy---
Success: no issues found in 2 source files
```

39. assistant (2026-06-21T07:26:46.120Z)

Lint and type checks are clean. Let me do a final review of the one genuine asymmetry I found (empty-string compute_target override) against the precedence intent, and confirm the warning-branch logic once more by re-reading the exact lines.

The mismatch-warning block (loader.py:156-163):
- `ct = merged["deployment"].get("compute_target")`
- `if ct and canonical and ct != canonical:` ? warns only for a *non-empty* divergent target.

The asymmetry is real but its impact is bounded: in M1, `compute_target` is inert (no consumer), so a silently-blanked compute_target under an active profile has zero behavioral effect this milestone. The honesty cost is that the observability field the loader stamps for the active profile can be silently emptied without the loud warning that a *wrong* value gets. This is a minor/low finding, not a blocker, and arguably acceptable since the field is documented as inert until M2+.

Everything else ? defaults<preset<config.json<env precedence, deep-merge sibling survival, env-wins re-stamp honesty, free-form dict merge (all default `{}` so merge==replace), and the no-profile path being byte-identical to the original `{**defaults, **file_data}` ? is correct and well-tested.

Let me record my findings.

40. user (2026-06-21T07:26:55.961Z)

Structured output provided successfully

41. assistant (2026-06-21T07:27:10.194Z)

I completed the M1 precedence & deep-merge review. Summary of what I verified and the single finding:

Precedence chain `defaults < preset < config.json < env(DEPLOYMENT_PROFILE)` holds across every combination I traced through live `load_config` runs:
- (a) A config.json sub-key overrides a preset sub-key while sibling preset sub-keys survive ? confirmed via deep-merge for `indexing` (detector_impl override, skip_ai_handoff survives) and cross-section (`storage.backend` preset survives while file sets `storage.gcs`).
- (b) The env-wins-over-file re-stamp is honest: `merged["deployment"]["profile"]` is set to the env-chosen profile (not the file's), and `compute_target` flows from the env profile's preset when the file doesn't override it (`backend/config/loader.py:152-155`).
- (c) The compute_target mismatch warning correctly fires for non-empty divergent overrides and honors them (D15), verified for both env-profile and file-only-compute_target cases.
- (d) `_deep_merge` into free-form dict fields (`storage.local/s3/gcs/gdrive`, `fusion.cue_flags`) is safe: every one defaults to `{}` (`backend/config/models.py:110,316-319`), so merge==replace ? no surprise. The only non-empty container defaults are lists (`court_green_lower`, `allowed_upload_extensions`), which `_deep_merge` treats as scalars (replace), matching shallow-path semantics.
- (e) The no-profile path is byte-identical to the original `{**defaults, **file_data}` ? proven by direct comparison against a simulated pre-M1 loader, including the unknown-env-profile reset-to-`""` fallback.

One finding (minor / medium): an explicit `compute_target: ""` in config.json under an active known profile silently blanks the profile's compute_target without the mismatch warning (`backend/config/loader.py:156-163`), because the `if ct and ...` guard short-circuits on the falsy empty string. Impact is bounded ? `compute_target` is inert in M1 ? but it's a latent observability/honesty gap to close before M2 wires it to a real ComputeBackend.

No blockers. The 52 config tests pass, and ruff + mypy are clean on the changed files. M1 meets the PURE / ADDITIVE / zero-core-change bar: with `deployment.profile=''` the loaded config is byte-identical to pre-M1.

Relevant files: `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\presets.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py`,

`C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_config_deployment.py`.